

✓ Predict and Forecast cashflows for suppliers and vendors using past data

In this notebook we will explore different ways to predict the cashflows and then narrow it down to one best method.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from pmdarima import auto_arima
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_squared_error
from statsmodels.tools.eval_measures import rmse
from prophet import Prophet
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from scipy.stats.mstats import winsorize
import plotly.express as px
import plotly.graph_objects as go
import warnings
warnings.filterwarnings('ignore')

# Load Data
sales_df = pd.read_csv("D:/MBA/SPJIMR/PlaceCom/Placements/Jd's/eFunderAI/Data Science Task/Data Science Task/sales.csv", thousands=",")
#datatype conversion

sales_df['date/time'] = pd.to_datetime(sales_df['date/time'])
sales_df['product_sales'] = sales_df.product_sales.astype(float)
```

✓ EDA

```
sales_df.head()
```



	date/time	settlement id	type	order id	quantity	fulfillment	product_sales	shipping_credits	promotional_rebates	selling_fe
0	2023-01-03 10:06:11+00:00	19018879082	Shipping Services	407- 2360018- 7202700	NaN	Seller	0.0	0.0	0.0	(
1	2023-01-03 10:06:24+00:00	19018879082	Shipping Services	171- 7449267- 3612317	NaN	Seller	0.0	0.0	0.0	(
2	2023-01-03 10:07:00+00:00	19018879082	Shipping Services	402- 2096374- 8628366	NaN	Seller	0.0	0.0	0.0	(
3	2023-01-03 10:07:03+00:00	19018879082	Shipping Services	406- 4531287- 8998701	NaN	Seller	0.0	0.0	0.0	(
4	2023-01-03 10:07:03+00:00	19018879082	Shipping Services	171- 4806703- 0631532	NaN	Seller	0.0	0.0	0.0	(

```
sales_df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10237 entries, 0 to 10236
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date/time              10237 non-null  datetime64[ns, UTC]
1   settlement id          10237 non-null  int64
2   type                  10237 non-null  object
3   order id              10148 non-null  object
4   quantity              9940 non-null   float64
5   fulfillment            10148 non-null  object
6   product_sales          10237 non-null  float64
```

```

7 shipping_credits      10237 non-null float64
8 promotional_rebates   10237 non-null float64
9 selling_fees          10237 non-null float64
10 fba_fees             10237 non-null float64
11 other_transaction_fees 10237 non-null float64
12 other                10237 non-null float64
13 total                10237 non-null float64
dtypes: datetime64[ns, UTC](1), float64(9), int64(1), object(3)
memory usage: 1.1+ MB

```

```

# Check for Missing Values
print("\nMissing Values:\n", sales_df.isnull().sum())

```



```

Missing Values:
date/time      0
settlement id  0
type           0
order id       89
quantity       297
fulfillment    89
product_sales  0
shipping_credits 0
promotional_rebates 0
selling_fees   0
fba_fees       0
other_transaction_fees 0
other          0
total          0
dtype: int64

```

```

# Summary Statistics
print("\nSales Statistics:\n", sales_df.describe())

```



```

Sales Statistics:
count      settlement id      quantity      product_sales      shipping_credits \
mean      2.192871e+10      1.299396      112.146255      4.705378
std       1.008402e+09      0.848582      139.016563      2.271347
min       1.901888e+10      1.000000      -940.380000      -14.000000
25%      2.122922e+10      1.000000      53.670000      5.000000
50%      2.185859e+10      1.000000      85.510000      5.000000
75%      2.275409e+10      1.000000      129.560000      5.000000
max       2.388629e+10      14.000000      2899.000000      19.560000

count      promotional_rebates      selling_fees      fba_fees \
mean      -0.042090      -15.897574      -0.398382
std       0.606163      19.511378      2.963105
min      -18.550000      -396.390000      -50.240000
25%      0.000000      -18.690000      0.000000
50%      0.000000      -12.250000      0.000000
75%      0.000000      -7.630000      0.000000
max       1.430000      112.610000      10.000000

count      other_transaction_fees      other      total
mean      -0.567928      -97.736965      2.208693
std       5.847032      1637.747297      1648.152763
min      -357.630000      -47387.140000      -47387.140000
25%      0.000000      0.000000      50.030000
50%      0.000000      0.000000      76.620000
75%      0.000000      0.000000      116.010000
max       0.000000      199.500000      2507.610000

```

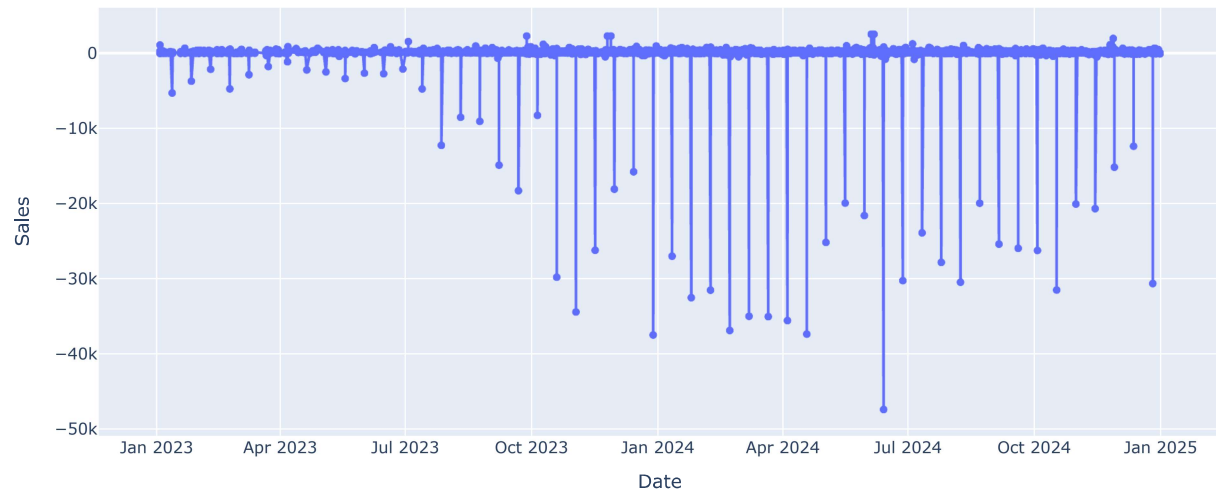
```

# Sales Trend Over Time
fig = px.line(sales_df, x="date/time", y="total", title="Sales Trend Over Time - Plot 1", markers=True)
fig.update_xaxes(title="Date")
fig.update_yaxes(title="Sales")
fig.update_layout(width=1000, height=500)
fig.show()

```



Sales Trend Over Time - Plot 1



All the big negatives are transfer payments - detailed analysis done in next part.

```
#aggregate sales data by week
week_df = sales_df.copy()
week_df['week_start'] = sales_df['date/time'].dt.to_period('W').apply(lambda r: r.start_time)
# Aggregate by week
weekly_summary = week_df.groupby('week_start').sum(numeric_only=True).reset_index()

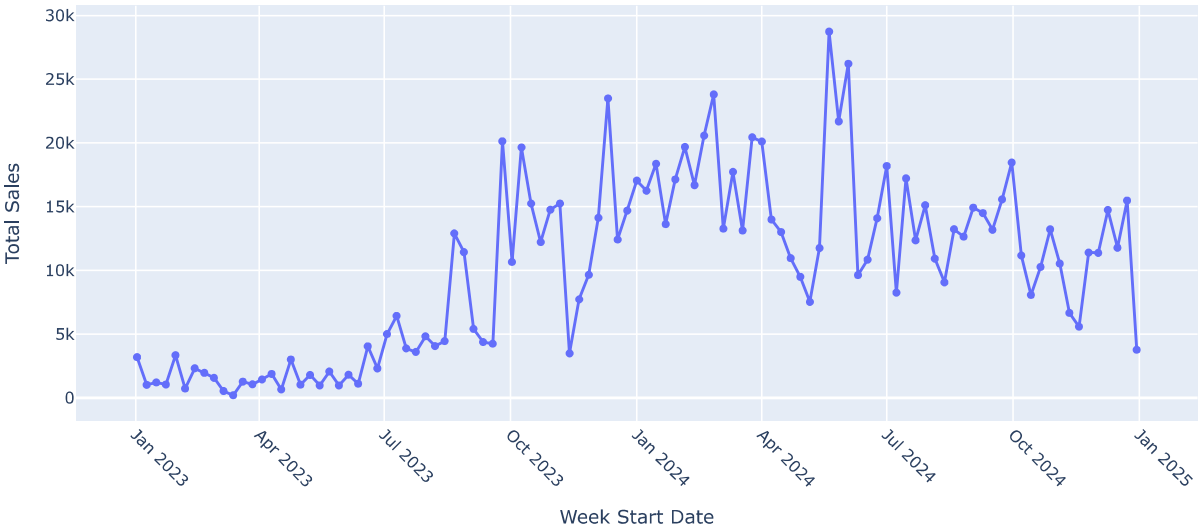
# Create the Plotly Line Chart
fig = px.line(weekly_summary, x="week_start", y="total",
              title="Total Sales vs Week - Plot 2", markers=True,
              labels={"week_start": "Week Start Date", "total": "Total Sales"})

# Update Layout for better readability
fig.update_xaxes(tickangle=45, showgrid=True)
fig.update_yaxes(showgrid=True)
fig.update_layout(width=1000, height=500)

# Show the interactive Plotly figure
fig.show()
```



Total Sales vs Week - Plot 2



▼ Demystifying Transfer payments

```
# Filtering the transfer payments transactions and calculating the time difference between two such transactions.
df_transfers = sales_df[sales_df['type']=='Transfer'].reset_index()
df_transfers['time_diff'] = np.nan
for row, val in df_transfers[1:].iterrows():
    df_transfers.loc[row, 'time_diff'] = df_transfers.loc[row-1, 'date/time'] - df_transfers.loc[row-1, 'date/time']
df_transfers.head()
```



	index	date/time	settlement_id	type	order_id	quantity	fulfillment	product_sales	shipping_credits	promotional_rebates	sellin
0	45	2023-01-12 06:27:19+00:00	19104707552	Transfer	NaN	NaN	NaN	0.0	0.0	0.0	
1	63	2023-01-26 06:19:02+00:00	19186506952	Transfer	NaN	NaN	NaN	0.0	0.0	0.0	
2	104	2023-02-09 06:18:42+00:00	19270185212	Transfer	NaN	NaN	NaN	0.0	0.0	0.0	
3	131	2023-02-23 05:59:20+00:00	19356292502	Transfer	NaN	NaN	NaN	0.0	0.0	0.0	
4	157	2023-03-09 05:54:28+00:00	19445854392	Transfer	NaN	NaN	NaN	0.0	0.0	0.0	

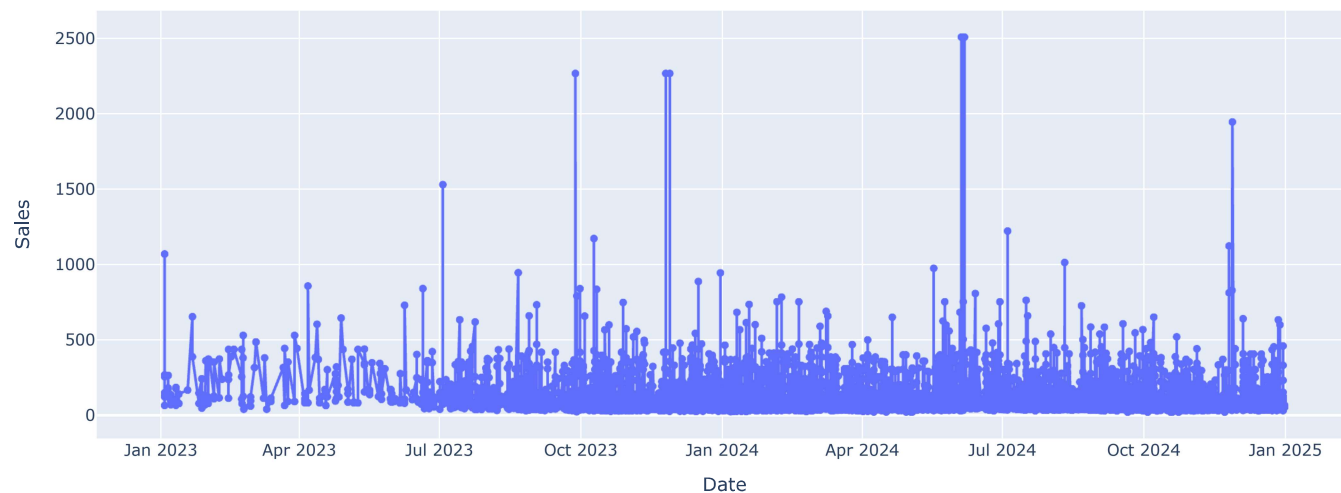
All the payments are made in an interval of 2 weeks as confirmed by time_diff field.

```
# Removing the transfer and other type of records as they don't actually count for sales.
sales_df = sales_df[sales_df['type']=='Order'].reset_index()

fig = px.line(sales_df, x="date/time", y="total", title="Sales Trend Over Time - Plot 3", markers=True)
fig.update_xaxes(title="Date")
fig.update_yaxes(title="Sales")
fig.update_layout(width=1100, height=500)
fig.show()
```



Sales Trend Over Time - Plot 3



Now all the payments are positive, although there are still some outlier payments.

```
#aggregating by week
week_df = sales_df.copy()
week_df['week_start'] = sales_df['date/time'].dt.to_period('W').apply(lambda r: r.start_time)
# Aggregate by week
weekly_summary = week_df.groupby('week_start').sum(numeric_only=True).reset_index()

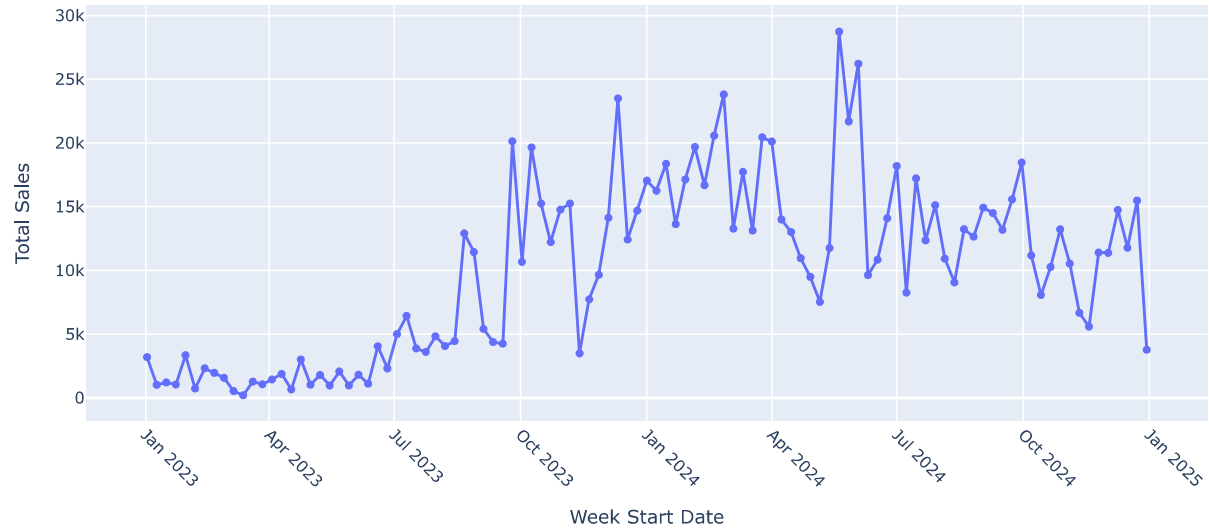
# Create the Plotly Line Chart
fig = px.line(weekly_summary, x="week_start", y="total",
              title="Total Sales vs Week - Plot 4", markers=True,
              labels={"week_start": "Week Start Date", "total": "Total Sales"})

# Update Layout for better readability
fig.update_xaxes(tickangle=45, showgrid=True)
fig.update_yaxes(showgrid=True)
fig.update_layout(width=1000, height=500)

# Show the interactive Plotly figure
fig.show()
```



Total Sales vs Week - Plot 4

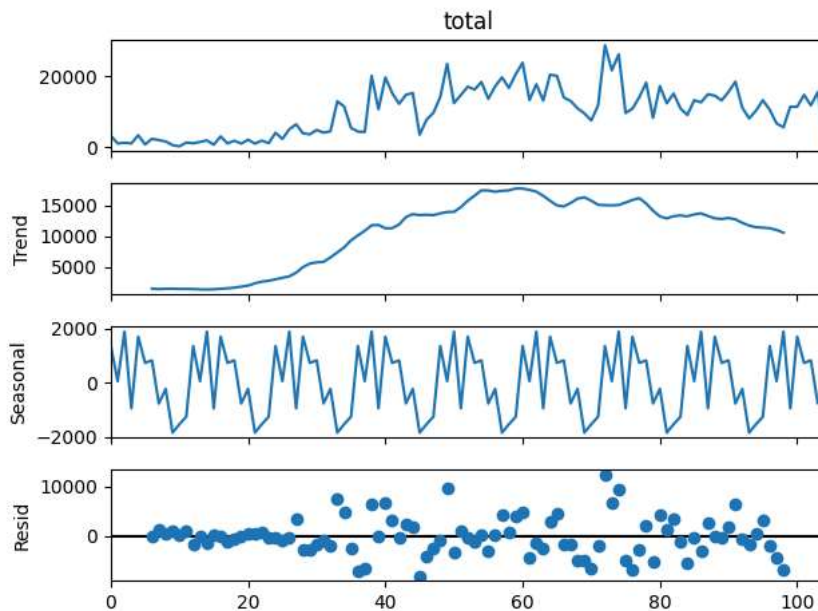


✓ Checking for Seasonality, ETS

```
# ETS Decomposition
result = seasonal_decompose(weekly_summary['total'], model='additive', period=12)
```

```
# ETS plot
result.plot()
print("Seasonality over time - Plot 5")
```

Seasonality over time - Plot 5



```
# confirming seasonality with ADF test
adf_test = adfuller(weekly_summary['total'])
```

```
print('ADF Statistic: %f' % adf_test[0])
print('p-value: %f' % adf_test[1])
```

ADF Statistic: -1.478953
p-value: 0.543896

As p-value > 0.05, the data contains seasonality

```
# Moving Average for Sales Trend (7-day and 30-day)
```

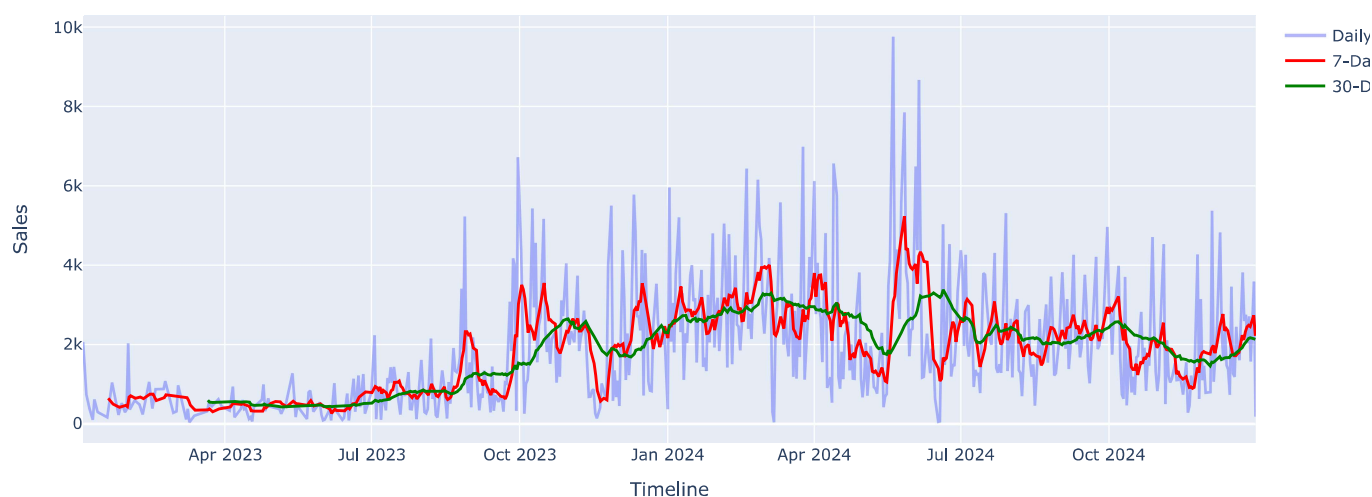
```
day_df = sales_df.copy()
day_df = day_df[day_df['type']=='Order']
day_df['day_start'] = day_df['date/time'].dt.date
# Aggregate by day
day_df = day_df.groupby('day_start').sum(numeric_only=True).reset_index()
day_df["sales_7d_MA"] = day_df["total"].rolling(window=7).mean()
day_df["sales_30d_MA"] = day_df["total"].rolling(window=30).mean()
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=day_df["day_start"], y=day_df["total"], mode="lines", name="Daily Sales", opacity=0.5))
fig.add_trace(go.Scatter(x=day_df["day_start"], y=day_df["sales_7d_MA"], mode="lines", name="7-Day MA", line=dict(color="red")))
fig.add_trace(go.Scatter(x=day_df["day_start"], y=day_df["sales_30d_MA"], mode="lines", name="30-Day MA", line=dict(color="green")))
```

```
fig.update_layout(title="Sales Trend with Moving Averages in Days - Plot 6", xaxis_title="Timeline", yaxis_title="Sales", width=1100, height=500)
fig.show()
```



Sales Trend with Moving Averages in Days - Plot 6



```
# Moving Average for Sales Trend (4-week and 12-week)
```

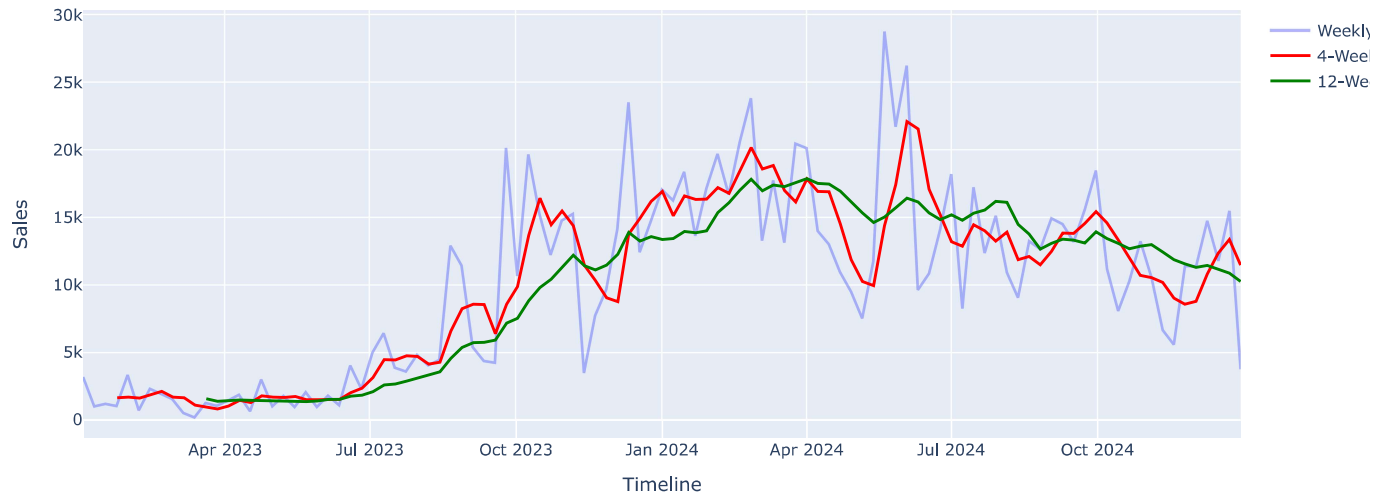
```
weekly_summary["sales_4w_MA"] = weekly_summary["total"].rolling(window=4).mean()
weekly_summary["sales_12w_MA"] = weekly_summary["total"].rolling(window=12).mean()
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=weekly_summary["week_start"], y=weekly_summary["total"], mode="lines", name="Weekly Sales", opacity=0.5))
fig.add_trace(go.Scatter(x=weekly_summary["week_start"], y=weekly_summary["sales_4w_MA"], mode="lines", name="4-Week MA", line=dict(color="red")))
fig.add_trace(go.Scatter(x=weekly_summary["week_start"], y=weekly_summary["sales_12w_MA"], mode="lines", name="12-Week MA", line=dict(color="green")))

fig.update_layout(title="Sales Trend with Moving Averages in Weeks - Plot 7", xaxis_title="Timeline", yaxis_title="Sales", width=1100, height=500)
fig.show()
```



Sales Trend with Moving Averages in Weeks - Plot 7



Insights : The 12 week Moving Average line resembles closely with the trends line in the seasonality graph.

✓ Model Building

✓ ARIMA (Weekly)

```
# Fit auto_arima function to Weekly sales dataset
stepwise_fit = auto_arima(weekly_summary['total'], start_p = 1, start_q = 1,
                          max_p = 3, max_q = 3, m = 12,
                          start_P = 0, seasonal = True,
                          d = None, D = 1, trace = True,
                          error_action = 'ignore', # we don't want to know if an order does not work
                          suppress_warnings = True, # we don't want convergence warnings
                          stepwise = True)

stepwise_fit.summary()
```



```

Performing stepwise search to minimize aic
ARIMA(1,1,1)(0,1,1)[12] : AIC=inf, Time=0.68 sec
ARIMA(0,1,0)(0,1,0)[12] : AIC=1914.525, Time=0.02 sec
ARIMA(1,1,0)(1,1,0)[12] : AIC=1873.816, Time=0.08 sec
ARIMA(0,1,1)(0,1,1)[12] : AIC=inf, Time=0.40 sec
ARIMA(1,1,0)(0,1,0)[12] : AIC=1901.625, Time=0.02 sec
ARIMA(1,1,0)(2,1,0)[12] : AIC=1869.041, Time=0.30 sec
ARIMA(1,1,0)(2,1,1)[12] : AIC=inf, Time=1.40 sec
ARIMA(1,1,0)(1,1,1)[12] : AIC=inf, Time=0.62 sec
ARIMA(0,1,0)(2,1,0)[12] : AIC=1865.452, Time=0.28 sec
ARIMA(0,1,0)(1,1,0)[12] : AIC=1882.924, Time=0.15 sec
ARIMA(0,1,0)(2,1,1)[12] : AIC=inf, Time=1.15 sec
ARIMA(0,1,0)(1,1,1)[12] : AIC=inf, Time=0.41 sec
ARIMA(0,1,1)(2,1,0)[12] : AIC=1845.551, Time=0.70 sec
ARIMA(0,1,1)(1,1,0)[12] : AIC=1853.333, Time=0.26 sec
ARIMA(0,1,1)(2,1,1)[12] : AIC=inf, Time=1.50 sec
ARIMA(0,1,1)(1,1,1)[12] : AIC=inf, Time=0.84 sec
ARIMA(1,1,1)(2,1,0)[12] : AIC=1842.842, Time=1.34 sec
ARIMA(1,1,1)(1,1,0)[12] : AIC=1853.351, Time=0.59 sec
ARIMA(1,1,1)(2,1,1)[12] : AIC=inf, Time=1.90 sec
ARIMA(1,1,1)(1,1,1)[12] : AIC=inf, Time=0.90 sec
ARIMA(2,1,1)(2,1,0)[12] : AIC=1863.682, Time=0.69 sec
ARIMA(1,1,2)(2,1,0)[12] : AIC=1863.014, Time=0.69 sec
ARIMA(0,1,2)(2,1,0)[12] : AIC=1861.270, Time=0.43 sec
ARIMA(2,1,0)(2,1,0)[12] : AIC=1856.083, Time=1.11 sec
ARIMA(2,1,2)(2,1,0)[12] : AIC=inf, Time=2.30 sec
ARIMA(1,1,1)(2,1,0)[12] intercept : AIC=1844.250, Time=2.09 sec

```

Best model: ARIMA(1,1,1)(2,1,0)[12]

Total fit time: 20.868 seconds

SARIMAX Results

Dep. Variable:	y	No. Observations:	105
Model:	SARIMAX(1, 1, 1)x(2, 1, [], 12)	Log Likelihood	-916.421
Date:	Mon, 10 Feb 2025	AIC	1842.842
Time:	09:04:34	BIC	1855.451
Sample:	0	HQIC	1847.931
	- 105		

Covariance Type: opg

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.2794	0.160	1.750	0.080	-0.033	0.592
ma.L1	-0.8255	0.079	-10.391	0.000	-0.981	-0.670
ar.S.L12	-0.8016	0.080	-9.974	0.000	-0.959	-0.644
ar.S.L24	-0.4378	0.127	-3.444	0.001	-0.687	-0.189
sigma2	2.553e+07	1.19e-09	2.15e+16	0.000	2.55e+07	2.55e+07

Ljung-Box (L1) (Q): 0.01 Jarque-Bera (JB): 2.85

Prob(Q): 0.91 Prob(JB): 0.24

Heteroskedasticity (H): 2.30 Skew: 0.21

Prob(H) (two-sided): 0.02 Kurtosis: 3.75

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

[2] Covariance matrix is singular or near-singular, with condition number 1.13e+32. Standard errors may be unstable.

```

# Split data into train / test sets
train = weekly_summary.iloc[:len(weekly_summary)-25]
test = weekly_summary.iloc[len(weekly_summary)-25:]

model = SARIMAX(weekly_summary['total'],
                order = (1, 1, 1),
                seasonal_order =(2, 1, 1, 12))

result = model.fit()
result.summary()

```



SARIMAX Results

Dep. Variable:

total

No. Observations:

105

Model:

SARIMAX(1, 1, 1)x(2, 1, 1, 12)

Log Likelihood

-909.446

Date:

Mon, 10 Feb 2025

AIC

1830.892

Time:

09:04:38

BIC

1846.023

Sample:

0

HQIC

1836.999

- 105

Covariance Type: opg

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.2340	0.156	1.501	0.133	-0.072	0.540
ma.L1	-0.7833	0.093	-8.379	0.000	-0.966	-0.600
ar.S.L12	-0.2506	0.114	-2.196	0.028	-0.474	-0.027
ar.S.L24	-0.1227	0.138	-0.889	0.374	-0.393	0.148
ma.S.L12	-0.9967	0.127	-7.842	0.000	-1.246	-0.748
sigma2	1.76e+07	7.29e-09	2.41e+15	0.000	1.76e+07	1.76e+07
Ljung-Box (L1) (Q):	0.00	Jarque-Bera (JB):	19.08			
Prob(Q):	0.99	Prob(JB):	0.00			
Heteroskedasticity (H):	1.95	Skew:	0.43			
Prob(H) (two-sided):	0.07	Kurtosis:	5.06			

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

[2] Covariance matrix is singular or near-singular, with condition number 6.48e+30. Standard errors may be unstable.

```
start = len(train)
end = len(train) + len(test) - 1

# Predictions for one-year against the test set
predictions = result.predict(len(train), end,
                             typ = 'levels').rename("Predictions")

# plot predictions and actual values
predictions.plot(legend = True)
weekly_summary['total'].plot(legend = True, figsize=(12,6), xlabel='Weeks', ylabel='Sales', title = 'SARIMA Predictions (Weekly) - Plot 8')

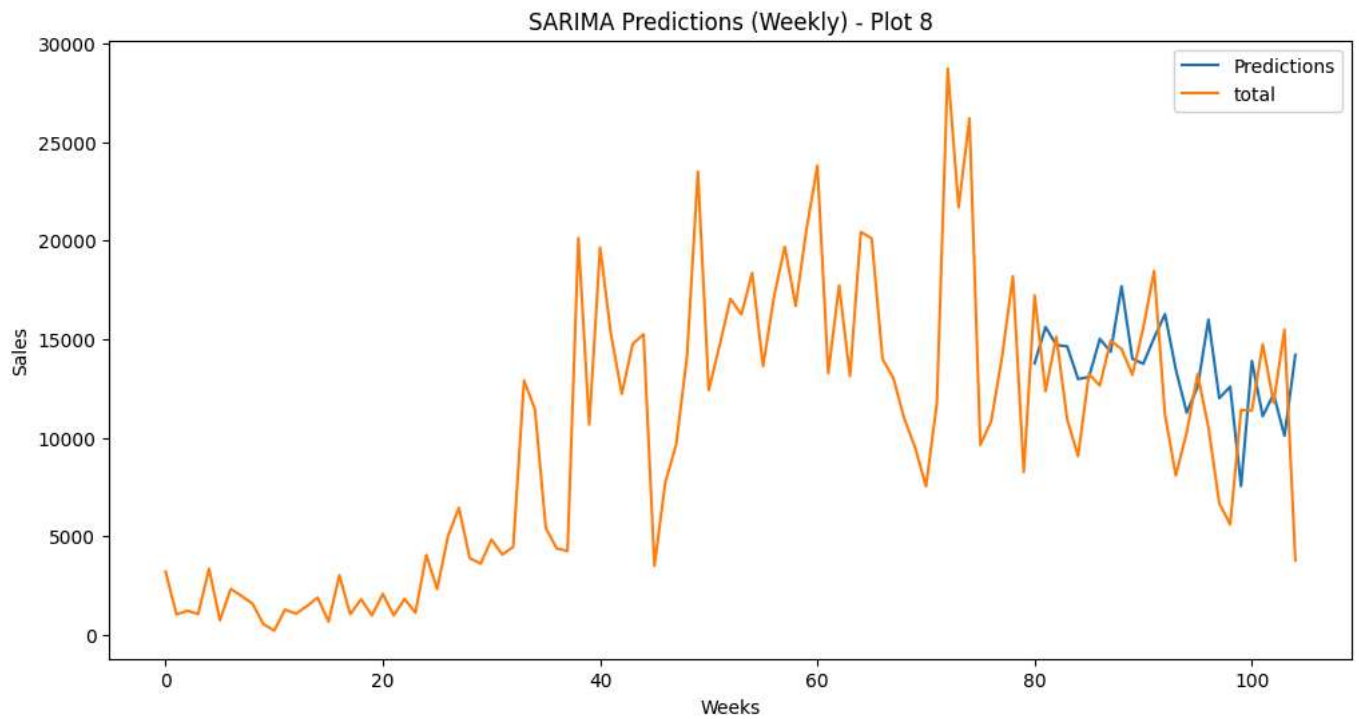
# Calculate root mean squared error
rmse(test["total"], predictions)

# Calculate mean squared error
mean_squared_error(test["total"], predictions)

mape = np.mean(np.abs((test['total'] - predictions) / test['total'])) * 100

print(f"MAPE: {mape: .2f}%")
```

MAPE: 39.51%



✓ ARIMA (Daily)

```
#aggregating by day
day_df = sales_df.copy()
day_df = day_df[day_df['type'] == 'Order']

day_df['day_start'] = sales_df['date/time'].dt.date
# Aggregate by week
day_df = day_df.groupby('day_start').sum(numeric_only=True).reset_index()
day_df_winsorize = day_df.copy()
day_df_winsorize['total'] = winsorize(day_df['total'], limits=[0.01, 0.01]) # Capping lowest & highest 1%

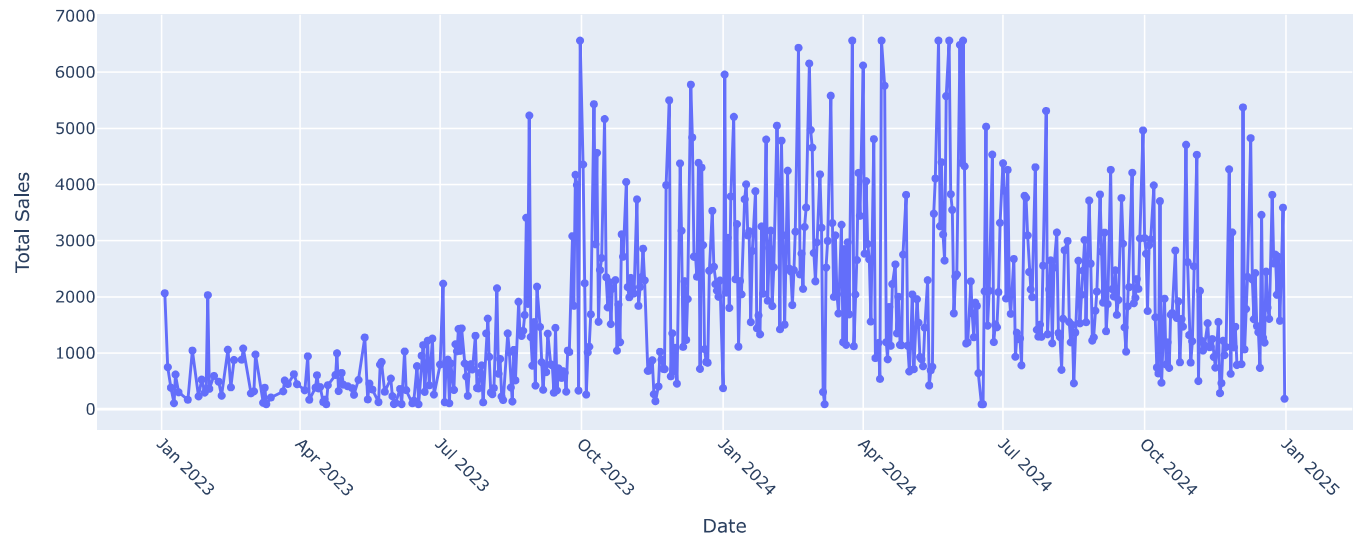
# Create the Plotly Line Chart
fig = px.line(day_df_winsorize, x="day_start", y="total",
              title="Total Sales vs Date - Plot 9", markers=True,
              labels={"day_start": "Date", "total": "Total Sales"})

# Update Layout for better readability
fig.update_layout(
    xaxis=dict(tickangle=45, showgrid=True),
    yaxis=dict(showgrid=True),
    legend_title="Legend",
    showlegend=True, width=1100, height=500
)

# Show the interactive Plotly figure
fig.show()
```



Total Sales vs Date - Plot 9



```
# Checking for Seasonality, ETS
```

```
# ETS Decomposition
```

```
result = seasonal_decompose(day_df_winsorize['total'], model='additive', period=90)
```

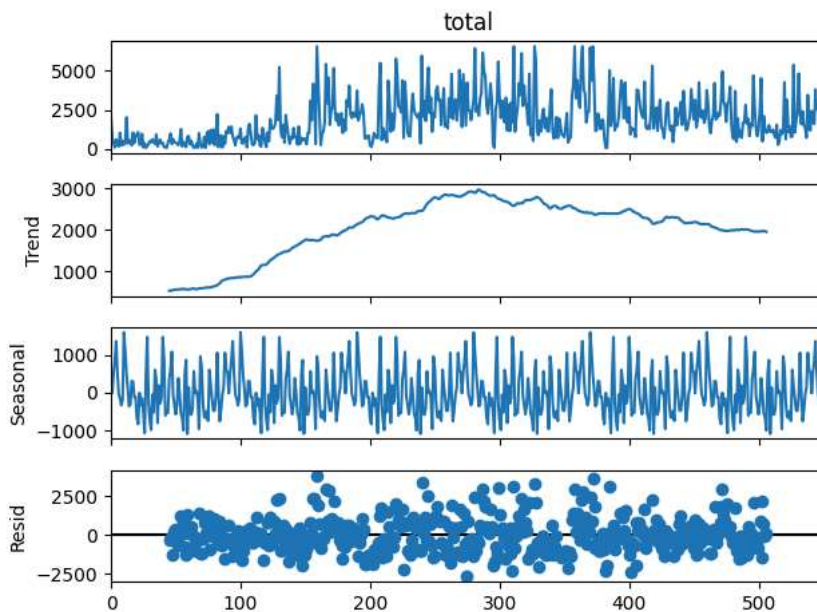
```
# ETS plot
```

```
result.plot()
```

```
print("Seasonality over time - Plot 10")
```



Seasonality over time - Plot 10



```
# Fit auto_arima function to AirPassengers dataset
```

```
stepwise_fit = auto_arima(day_df_winsorize['total'], start_p = 1, start_q = 1,
```

```
max_p = 3, max_q = 3, m = 12,
```

```
start_P = 0, seasonal = True,
```

```
d = None, D = 1, trace = True,
```

```
error_action = 'ignore', # we don't want to know if an order does not work
```

```
suppress_warnings = True, # we don't want convergence warnings
```

```
stepwise = True)
```

```
stepwise_fit.summary()
```

```
➦ Performing stepwise search to minimize aic
ARIMA(1,0,1)(0,1,1)[12] intercept : AIC=9185.004, Time=2.19 sec
ARIMA(0,0,0)(0,1,0)[12] intercept : AIC=9471.412, Time=0.04 sec
ARIMA(1,0,0)(1,1,0)[12] intercept : AIC=9315.078, Time=0.90 sec
ARIMA(0,0,1)(0,1,1)[12] intercept : AIC=9225.732, Time=1.56 sec
ARIMA(0,0,0)(0,1,0)[12] intercept : AIC=9469.768, Time=0.04 sec
ARIMA(1,0,1)(0,1,0)[12] intercept : AIC=9400.824, Time=0.17 sec
ARIMA(1,0,1)(1,1,1)[12] intercept : AIC=inf, Time=3.33 sec
ARIMA(1,0,1)(0,1,2)[12] intercept : AIC=9181.196, Time=7.19 sec
ARIMA(1,0,1)(1,1,2)[12] intercept : AIC=inf, Time=8.69 sec
ARIMA(0,0,1)(0,1,2)[12] intercept : AIC=9225.752, Time=5.15 sec
ARIMA(1,0,0)(0,1,2)[12] intercept : AIC=9209.134, Time=7.72 sec
ARIMA(2,0,1)(0,1,2)[12] intercept : AIC=9190.175, Time=10.83 sec
ARIMA(1,0,2)(0,1,2)[12] intercept : AIC=inf, Time=10.39 sec
ARIMA(0,0,0)(0,1,2)[12] intercept : AIC=9259.513, Time=2.45 sec
ARIMA(0,0,2)(0,1,2)[12] intercept : AIC=9201.009, Time=7.48 sec
ARIMA(2,0,0)(0,1,2)[12] intercept : AIC=9187.579, Time=4.93 sec
ARIMA(2,0,2)(0,1,2)[12] intercept : AIC=inf, Time=10.76 sec
ARIMA(1,0,1)(0,1,2)[12] intercept : AIC=9181.751, Time=5.16 sec
```

Best model: ARIMA(1,0,1)(0,1,2)[12] intercept
Total fit time: 88.999 seconds

SARIMAX Results			
Dep. Variable:	y	No. Observations:	552
Model:	SARIMAX(1, 0, 1)x(0, 1, [1, 2], 12)	Log Likelihood	-4584.598
Date:	Mon, 10 Feb 2025	AIC	9181.196
Time:	09:06:48	BIC	9206.946
Sample:	0	HQIC	9191.267
	- 552		

Covariance Type: opg						
	coef	std err	z	P> z	[0.025	0.975]
intercept	5.1636	3.565	1.448	0.147	-1.823	12.151
ar.L1	0.8619	0.038	22.570	0.000	0.787	0.937
ma.L1	-0.6382	0.054	-11.736	0.000	-0.745	-0.532
ma.S.L12	-0.7364	0.036	-20.303	0.000	-0.807	-0.665
ma.S.L24	-0.1242	0.036	-3.409	0.001	-0.196	-0.053
sigma2	1.36e+06	6.63e+04	20.509	0.000	1.23e+06	1.49e+06
Ljung-Box (L1) (Q):	0.00	Jarque-Bera (JB):	132.44			
Prob(Q):	0.95	Prob(JB):	0.00			
Heteroskedasticity (H):	1.66	Skew:	0.71			
Prob(H) (two-sided):	0.00	Kurtosis:	4.97			

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
# Split data into train / test sets
train = day_df_winsorize.iloc[:len(day_df_winsorize)-116]
test = day_df_winsorize.iloc[len(day_df_winsorize)-116:]

model = SARIMAX(day_df_winsorize['total'],
                 order = (1, 0, 1),
                 seasonal_order =(0, 1, 1, 12))

result = model.fit()
result.summary()
```



SARIMAX Results

Dep. Variable:	total	No. Observations:	552
Model:	SARIMAX(1, 0, 1)x(0, 1, 1, 12)	Log Likelihood	-4588.328
Date:	Mon, 10 Feb 2025	AIC	9184.655
Time:	09:07:44	BIC	9201.821
Sample:	0	HQIC	9191.369
	- 552		

Covariance Type: opg

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.8597	0.038	22.600	0.000	0.785	0.934
ma.L1	-0.6447	0.052	-12.283	0.000	-0.748	-0.542
ma.S.L12	-0.7882	0.021	-36.859	0.000	-0.830	-0.746
sigma2	1.389e+06	5.93e+04	23.435	0.000	1.27e+06	1.5e+06
Ljung-Box (L1) (Q):	0.00	Jarque-Bera (JB):	144.94			
Prob(Q):	0.96	Prob(JB):	0.00			
Heteroskedasticity (H):	1.59	Skew:	0.67			
Prob(H) (two-sided):	0.00	Kurtosis:	5.15			

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
start = len(train)
end = len(train) + len(test) - 1

# Predictions for one-year against the test set
predictions = result.predict(len(train), end,
                             typ = 'levels').rename("Predictions")

# plot predictions and actual values
predictions.plot(legend = True)
day_df_winsorize['total'].plot(legend = True, figsize=(12,6), xlabel='Days', ylabel='Sales', title = 'SARIMA Predictions (Daily) - Plot 11')

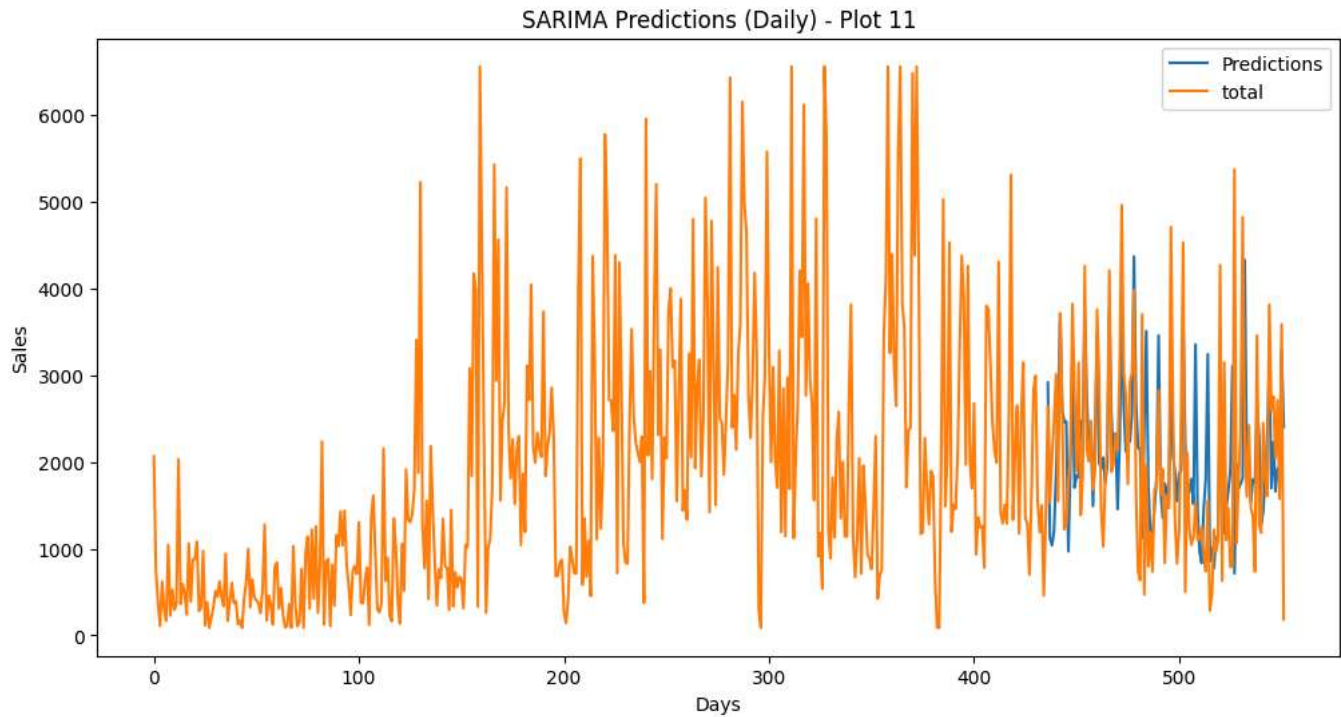
# Calculate root mean squared error
rmse(test["total"], predictions)

# Calculate mean squared error
mean_squared_error(test["total"], predictions)

mape = np.mean(np.abs((test['total'] - predictions) / test['total'])) * 100
print("MAPE", mape)
#60
```



MAPE 61.848255122785524



▼ Holt-Winters (Triple Exponential Smoothing)

```
# Apply Holt-Winters (Triple Exponential Smoothing)
model = ExponentialSmoothing(train['total'], trend="add", seasonal="add", seasonal_periods=90) # Weekly seasonality assumed
model_fit = model.fit(optimized=True, remove_bias=True)

# Forecast Future Values
forecast = model_fit.forecast(steps=len(test['total']))
# Create figure
fig = go.Figure()

# Add training data
fig.add_trace(go.Scatter(x=train.index, y=train['total'],
                        mode='lines', name="Training Data",
                        line=dict(color="blue"), opacity=0.8))

# Add actual cash flow
fig.add_trace(go.Scatter(x=test.index, y=test['total'],
                        mode='lines', name="Actual Cash Flow",
                        line=dict(color="green"), opacity=0.7))

# Add predicted cash flow
fig.add_trace(go.Scatter(x=test.index, y=forecast,
                        mode='lines', name="Predicted Cash Flow",
                        line=dict(color="red"), opacity=0.5))

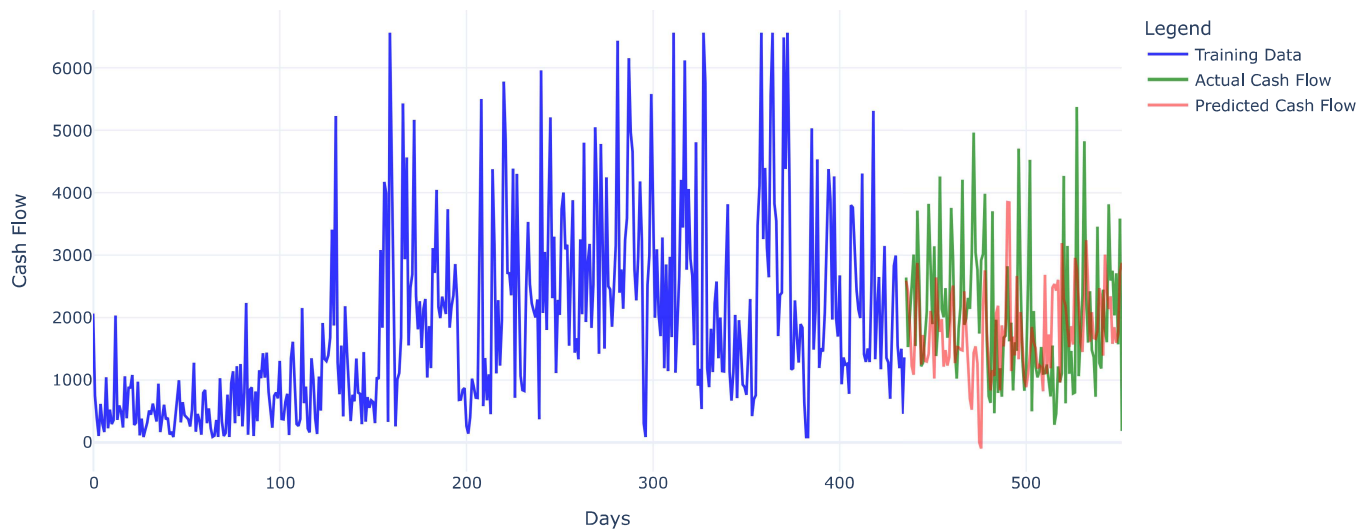
# Update layout
fig.update_layout(
    title="Holt-Winters Cash Flow Predictions (Day-wise) - Plot 12",
    xaxis_title="Days",
    yaxis_title="Cash Flow",
    legend_title="Legend",
    template="plotly_white"
)

# Show plot
fig.show()
#rmse(test, forecast)
mape = np.mean(np.abs((test['total'] - forecast) / test['total'])) * 100

print("MAPE", mape)
```



Holt-Winters Cash Flow Predictions (Day-wise) - Plot 12



MAPE 75.09629204251648

```
# Create figure
fig = go.Figure()
```



```

# Add actual cash flow line
fig.add_trace(go.Scatter(x=test.index, y=test['total'],
                        mode='lines', name="Actual Cash Flow",
                        line=dict(color="black"))))

# Add Holt-Winters forecast line
fig.add_trace(go.Scatter(x=test.index, y=forecast,
                        mode='lines', name="Holt-Winters",
                        line=dict( color="blue"),opacity=0.5))

# Add ARIMA forecast line
fig.add_trace(go.Scatter(x=test.index, y=predictions,
                        mode='lines', name="ARIMA",
                        line=dict(color="red"), opacity=0.5))

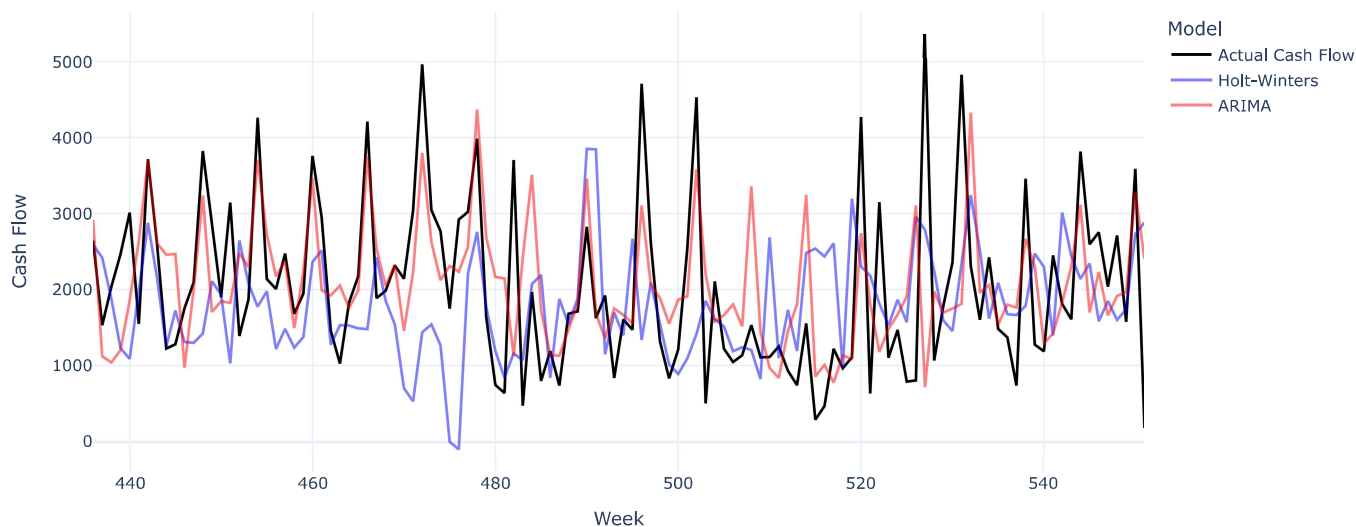
# Update layout
fig.update_layout(
    title="Cash Flow Prediction Comparison - Plot 13",
    xaxis_title="Week",
    yaxis_title="Cash Flow",
    legend_title="Model",
    template="plotly_white"
)

# Show plot
fig.show()

```



Cash Flow Prediction Comparison - Plot 13



✓ XGBoost (Daily)

```

import xgboost as xgb
from sklearn.metrics import mean_absolute_error, mean_squared_error
import statistics
from sklearn import metrics

```

```

def create_transaction_features(data):
    data = data.copy()
    data['day_start'] = pd.to_datetime(data['day_start'])
    # Time-based features
    #data['hour'] = data['date/time'].dt.hour
    data['dayofweek'] = data['day_start'].dt.dayofweek
    data['day'] = data['day_start'].dt.day
    data['month'] = data['day_start'].dt.month
    data['year'] = data['day_start'].dt.year
    data['is_weekend'] = (data['dayofweek'] >= 5).astype(int)

    # Lag Features (Using previous transaction values)
    for lag in [1, 7, 14, 21, 28, 56, 84, 112]: # Different lags for short & long-term patterns

```



```

data[f'lag_{lag}'] = data['total'].shift(lag)

# Rolling window features (captures recent trends)
data['rolling_mean_5'] = data['total'].rolling(window=5).mean()
data['rolling_std_5'] = data['total'].rolling(window=5).std()

return data.dropna() # Drop initial rows with NaN due to shifting

df = sales_df.copy()
df = df[df['type']=='Order']
df['day_start'] = df['date/time'].dt.date
# Aggregate by day
df = df.groupby('day_start').sum(numeric_only=True).reset_index()
df = create_transaction_features(df)

df['total'] = winsorize(df['total'], limits=[0.01, 0.01]) # Capping lowest & highest 1%

df = df.drop(['index', 'settlement id', 'quantity', 'product_sales', 'shipping_credits',
'promotional_rebates', 'selling_fees', 'fba_fees',
'other_transaction_fees', 'other', 'day_start' ], axis=1)
#df.drop(df.tail(1).index,inplace=True)

# ----- Train-Test Split ----- #
train_size = int(len(df) * 0.8)
train, test = df.iloc[:train_size], df.iloc[train_size:]

X_train, y_train = train.drop(columns=[ 'total']), train['total']
X_test, y_test = test.drop(columns=[ 'total']), test['total']

```

X_train.head()

	dayofweek	day	month	year	is_weekend	lag_1	lag_7	lag_14	lag_21	lag_28	lag_56	lag_84	lag_112	rolling_mean_5	rolling_
112	0	7	8	2023	0	376.26	121.30	799.92	1428.32	845.29	457.02	207.18	2064.73	1264.425	1256.0
113	1	8	8	2023	0	2152.59	1348.16	707.03	1039.08	882.49	349.37	317.53	747.55	1391.740	1076.0
114	2	9	8	2023	0	630.89	1612.94	1306.15	1438.26	105.31	125.59	511.05	383.06	762.850	186.6
115	3	10	8	2023	0	894.81	930.92	377.38	813.53	815.97	794.27	446.57	106.13	559.725	473.8
116	4	11	8	2023	0	224.64	296.49	371.51	580.11	342.95	840.69	622.57	617.54	193.430	44.1

```

# ----- Train XGBoost Model ----- #
model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=100, learning_rate=0.05, max_depth=10)
model.fit(X_train, y_train)

```

```

XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.05, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=10, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=100, n_jobs=None,
              num_parallel_tree=None, random_state=None, ...)

```

y_pred = model.predict(X_test)

```

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
test.rename(columns={'dayofmonth': 'day'}, inplace=True)

```

```

print(f"XGBoost - MAE: {mae:.2f}, RMSE: {rmse:.2f}, MAPE: {mape:.2f}%")
test['date/time'] = pd.to_datetime(test[['year', 'month', 'day']])

```

fig = go.Figure()

```
# Add actual cash flow
fig.add_trace(go.Scatter(x=test['date/time'], y=y_test,
                        mode='lines', name="Actual Cash Flow",
                        line=dict(color="black"), opacity=0.8))

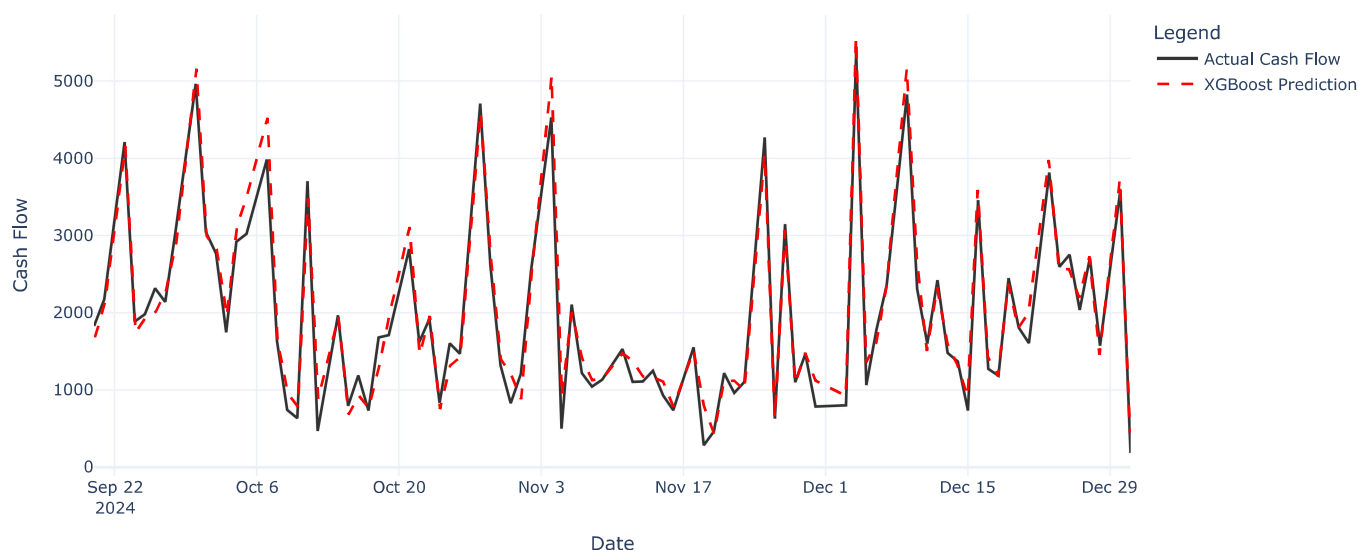
# Add XGBoost predictions
fig.add_trace(go.Scatter(x=test['date/time'], y=y_pred,
                        mode='lines', name="XGBoost Prediction",
                        line=dict(dash="dash", color="red"))))

# Update layout
fig.update_layout(
    title="XGBoost Cash Flow Prediction (Day Level) - Plot 14",
    xaxis_title="Date",
    yaxis_title="Cash Flow",
    legend_title="Legend",
    template="plotly_white"
)

# Show plot
fig.show()
```

🔄 XGBoost - MAE: 163.45, RMSE: 210.28, MAPE: 14.48%

XGBoost Cash Flow Prediction (Day Level) - Plot 14



XGBoost is giving quite good predictions for the first attempt, with MAPE of just 14.48%.

Areas to improve in the model:

1. If we look at the whole data aggregated daily, then the last data point is an outlier. Train the model without it.
2. Fine tune the Hyper-parameters using GridSearchCV
3. Forecast the next 30 days sales and then compare it with the actual data when 30 days are crossed.
4. Based on the other data available calculate the risk and adjust the predicted total sales to get a more realistic predictions.

Implementing 1. If we look at the whole data aggregated daily, then the last data point is an outlier. Train the model without it. 2. Fine tune the Hyper-parameters using GridSearchCV

```
df.drop(df.tail(1).index,inplace=True)
# ----- Train-Test Split ----- #
train_size = int(len(df) * 0.8)
train, test = df.iloc[:train_size], df.iloc[train_size:]

X_train, y_train = train.drop(columns=[ 'total' ]), train['total']
X_test, y_test = test.drop(columns=[ 'total' ]), test['total']

# ----- Train XGBoost Model ----- #
model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=100, learning_rate=0.05, max_depth=10)
```

```

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
test.rename(columns={'dayofmonth': 'day'}, inplace=True)

print(f"XGBoost - MAE: {mae:.2f}, RMSE: {rmse:.2f}, MAPE: {mape:.2f}%")
test['date/time'] = pd.to_datetime(test[['year', 'month', 'day']])

fig = go.Figure()

# Add actual cash flow
fig.add_trace(go.Scatter(x=test['date/time'], y=y_test,
                        mode='lines', name="Actual Cash Flow",
                        line=dict(color="black"), opacity=0.8))

# Add XGBoost predictions
fig.add_trace(go.Scatter(x=test['date/time'], y=y_pred,
                        mode='lines', name="XGBoost Prediction",
                        line=dict(dash="dash", color="red"))))

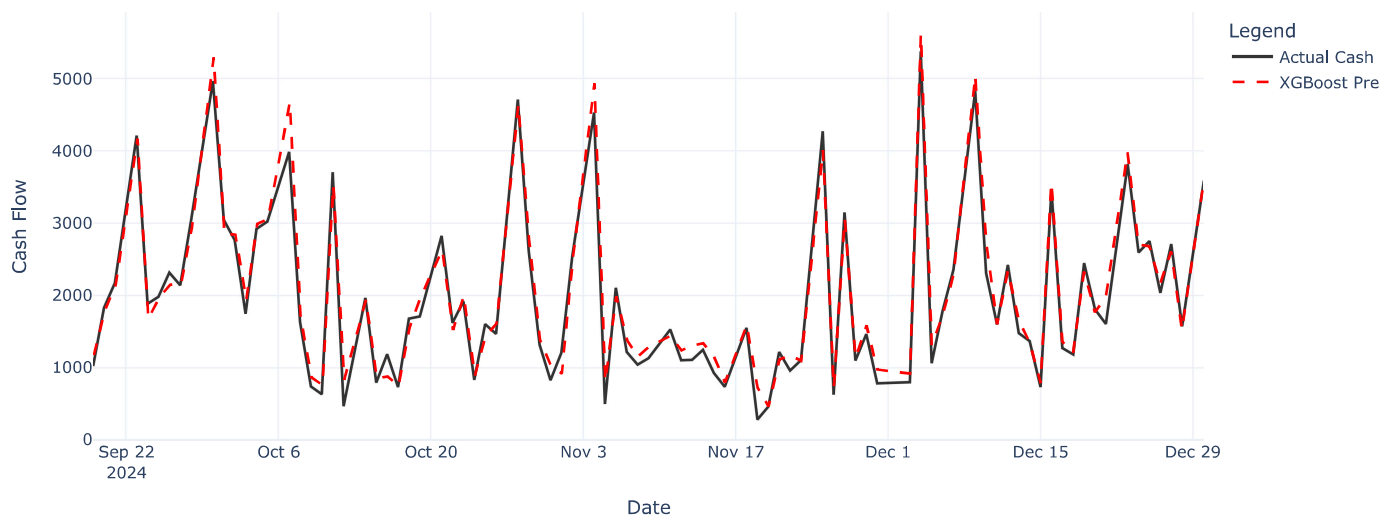
# Update layout
fig.update_layout(
    title="XGBoost Cash Flow Prediction (Day Level) - Plot 15",
    xaxis_title="Date",
    yaxis_title="Cash Flow",
    legend_title="Legend",
    template="plotly_white", width=1100, height=500
)

# Show plot
fig.show()

```

🔗 XGBoost - MAE: 138.42, RMSE: 181.52, MAPE: 10.87%

XGBoost Cash Flow Prediction (Day Level) - Plot 15



Using GridSearchCV

```

from sklearn.model_selection import GridSearchCV

# A parameter grid for XGBoost
params = {
    'min_child_weight': [4, 5],
    'gamma': [i / 10.0 for i in range(3, 6)],
    'subsample': [i / 10.0 for i in range(6, 11)],
    'colsample_bytree': [i / 10.0 for i in range(6, 11)],
    'max_depth': [2, 3, 4, 5]
}

```

```

}
# Initialize XGB and GridSearch
xgb_reg = xgb.XGBRegressor(nthread=-1, objective='reg:squarederror')
grid = GridSearchCV(xgb_reg, params)
grid.fit(X_train, y_train)
gridcv_xgb = grid.best_estimator_

```

```

print(metrics.r2_score(y_test, gridcv_xgb.predict(X_test)))
from xgboost import plot_importance, plot_tree

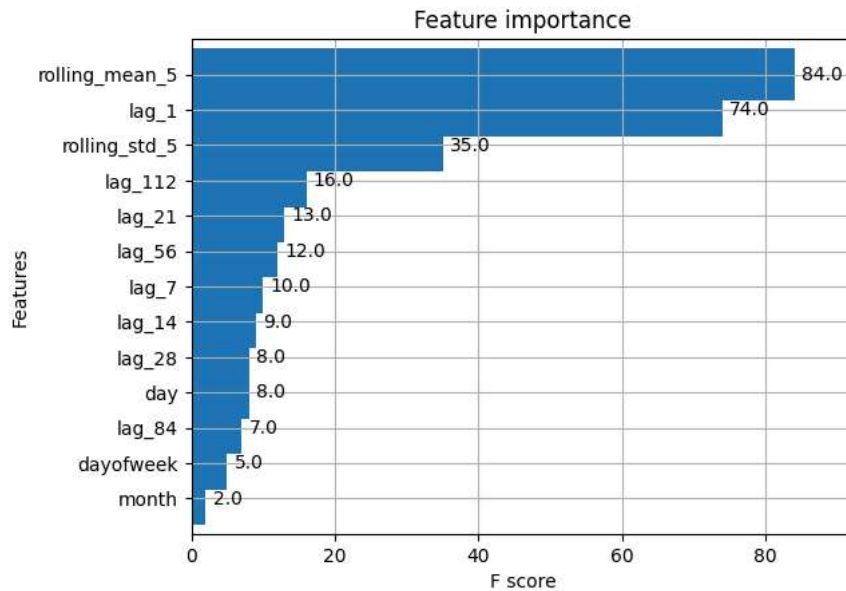
```

```

_ = plot_importance(gridcv_xgb, height=1.5)
print("Feature Importance Scores - Plot 16")

```

↗ 0.9713615205300865
Feature Importance Scores - Plot 16



Rolling Mean captures the most variance of the data followed by lag_1 and then rolling std-dev.

```

y_pred_GridSearchCV = gridcv_xgb.predict(X_test)
mae = mean_absolute_error(y_test, y_pred_GridSearchCV)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_GridSearchCV))
mape = np.mean(np.abs((y_test - y_pred_GridSearchCV) / y_test)) * 100
test.rename(columns={'dayofmonth': 'day'}, inplace=True)

print(f"XGBoost - MAE: {mae:.2f}, RMSE: {rmse:.2f}, MAPE: {mape:.2f}%")
test['date/time'] = pd.to_datetime(test[['year', 'month', 'day']])

fig = go.Figure()

# Add actual cash flow
fig.add_trace(go.Scatter(x=test['date/time'], y=y_test,
                        mode='lines', name="Actual Cash Flow",
                        line=dict(color="black"), opacity=0.8))

# Add XGBoost predictions
fig.add_trace(go.Scatter(x=test['date/time'], y=y_pred_GridSearchCV,
                        mode='lines', name="XGBoost Prediction",
                        line=dict(dash="dash", color="red"))))

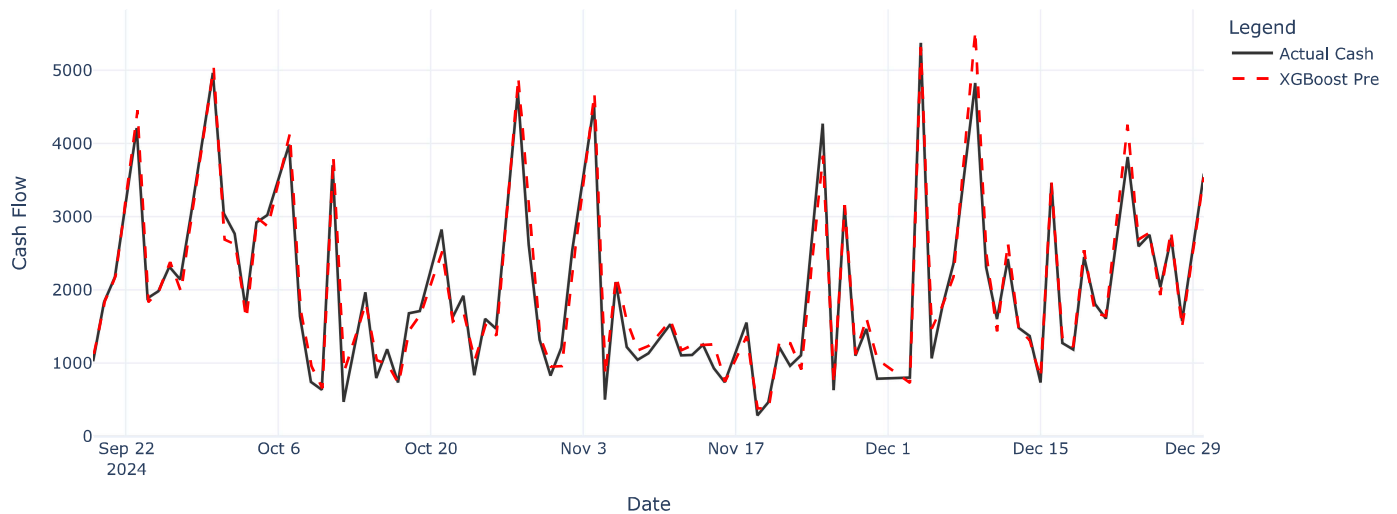
# Update layout
fig.update_layout(
    title="XGBoost Cash Flow Prediction (Day Level) GridSearchCV - Plot 17",
    xaxis_title="Date",
    yaxis_title="Cash Flow",
    legend_title="Legend",
    template="plotly_white", width=1100, height=500
)

# Show plot
fig.show()

```

XGBoost - MAE: 147.72, RMSE: 197.41, MAPE: 10.76%

XGBoost Cash Flow Prediction (Day Level) GridSearchCV - Plot 17



✓ Calculating factors for Risk Adjusted Sales

```
violation_df = pd.read_csv("D:/MBA/SPJIMR/PlaceCom/Placements/Jd's/eFunderAI/Data Science Task/Data Science Task/Product Policies Violator
performance_df = pd.read_csv("D:/MBA/SPJIMR/PlaceCom/Placements/Jd's/eFunderAI/Data Science Task/Data Science Task/Performance Over Time.c
inventory_df = pd.read_csv("D:/MBA/SPJIMR/PlaceCom/Placements/Jd's/eFunderAI/Data Science Task/Data Science Task/Inventory.csv")
```

```
def compute_inventory_risk(inventory_df):
    """
    Compute inventory risk factor based on stock levels, damages, and disposal.
    """
    inventory_df['Date'] = pd.to_datetime(inventory_df['Date'])
    three_months_ago = pd.to_datetime("today") - pd.DateOffset(months=5)
    recent_violations = inventory_df[inventory_df["Date"] >= three_months_ago]
    stock_risk = recent_violations['Vendor Returns'].sum()/recent_violations['Receipts'].sum()
    # Aggregate risk score

    return stock_risk

def compute_performance_risk(performance_df):
    #taking hte last 3 months performance issue ratio to the total orders
    performance_df['total'] = performance_df['Pre-fulfilment Cancellations'] + performance_df['Late Shipments'] + performance_df['Order Del
    performance_risk = performance_df[:3]['total'].sum()/performance_df[:3]['Total Orders'].sum()
    return abs(performance_risk)

def compute_voiolation_risk(violation_df):
    #Arbitrarily assigning the severity depending upon the consequences
    severity_dict = {
        "Product Condition Complaint - Wrong Product": 2,
        "Restricted products policy violation": 3,
        "Product Recall": 5,
        "Potential trademark misuse": 4,
        "Violation of Fair Pricing Policy": 3
    }
    violation_df['Date'] = pd.to_datetime(violation_df['Date'])
    violation_df['Severity_Score'] = violation_df['Product Violation'].map(severity_dict)
    # Filter violations within the last 3 months
    three_months_ago = pd.to_datetime("today") - pd.DateOffset(months=5)
    recent_violations = violation_df[violation_df["Date"] >= three_months_ago]

    # Sum of severity scores in the last 3 months
    total_severity_last_3_months = recent_violations["Severity_Score"].sum()

    return total_severity_last_3_months/90
```

```

performance_risk_factor = compute_performance_risk(performance_df)
inventory_risk_factor = compute_inventory_risk(inventory_df)
violation_risk_factor = compute_violation_risk(violation_df)
#Risk-Adjusted Sales=(Base Sales Forecast)x(1-Inventory Risk Factor)x(1-Product Violation Risk Factor)x(1-Performance Risk Factor)
v_pred_adjusted = v_pred*(1-performance_risk_factor)*(1-0.1*inventory_risk_factor)*(1-violation_risk_factor)

fig = go.Figure()

# Add actual cash flow
fig.add_trace(go.Scatter(x=test['date/time'], y=y_test,
                        mode='lines', name="Actual Cash Flow",
                        line=dict(color="black"), opacity=0.8))

# Add XGBoost predictions
fig.add_trace(go.Scatter(x=test['date/time'], y=y_pred_GridSearchCV,
                        mode='lines', name="XGB Prediction",
                        line=dict(dash="dash", color="red"))))

# Add XGBoost risk Adjusted predictions
fig.add_trace(go.Scatter(x=test['date/time'], y=y_pred_adjusted,
                        mode='lines', name="XGB Risk Adjusted Prediction",
                        line=dict(color="blue"), opacity=0.4))

# Update layout
fig.update_layout(
    title="XGBoost Cash Flow Prediction (Day Level) - Plot 18",

```